2026 International Conference on Smart Futuristic Technology (ICSFT)
Bangalore, India. Jan 2-3, 2026

PRISM: Prompt Red-teaming and Injection Simulation for Models – A Scalable Framework for Evaluating LLM Robustness Against Malicious Prompts

Nishith P
Dept. of Computer Science and Engineering
Amrita School of Computing, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.u4cse22138@bl.students.amrita.edu

Niharika Panda
Dept. of Computer Science and Engineering
Amrita School of Computing, Bengaluru
Amrita Vishwa Vidyapeetham, India
p_niharika@blr.amrita.edu

2026 International Conference on Smart Futuristic Technology (ICSFT) | 979-8-3503-5707-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICSFT66733.2026.11508065

***Abstract*—The rapid growth in the adoption of Large Language Models has brought with it some significant security vulnerabilities-one of which is prompt injection attacks. These attacks manipulate the behavior of LLMs using malicious user input, leading to the compromise of system integrity and the production of unintended or harmful output. Traditional detection approaches often cannot keep pace with the complexity and diversity of such attacks. We address this gap by presenting PRISM-Prompt Red-teaming and Injection Simulation for Models-a scalable framework designed to comprehensively assess the robustness of LLMs against adversarial prompts in a systematic manner. PRISM leverages a distributed computing architecture, utilizing Hadoop MapReduce and Ollama, to automatically assess LLM responses against a wide range of prompt variations, along with temperature configurations. It computes similarity-based measures from the large corpus of known prompt injection attempts that PRISM will process to describe model performance and identify successful attack vectors. The proposed work outlines the architecture of PRISM, reports on some initial exploratory data analysis of the HackAPrompt dataset, and demonstrates the ability of the framework to provide a complete, scalable, and automated solution for both LLM security auditing and red-teaming.**

***Index Terms*—Large Language Models, LLM Security, Prompt Injection, Red-Teaming, Hadoop, Big Data, Cybersecurity, Model Robustness.**

I. Introduction

LLMs have emerged as a transformative technology, substantially reordering the landscape of human–computer interaction and artificial intelligence [1]. Their ability to understand, generate, and reason about natural language has enabled their use in applications ranging from customer-service chatbots all the way to complex decision-support systems. However, broad deployment has revealed an important security vulnerability: the potential for prompt injection attacks [2].

A prompt injection attack occurs when an adversary crafts an input-the "prompt"-that overrides the LLM's original intent and causes the model to carry out unintended actions [3]. These include issues ranging from generating inappropriate content to leaking sensitive information and executing harmful code, thereby posing significant threats to system security and user trust. The insider nature and contextual dependence of these attacks inherently makes defense much more difficult. Traditional defense mechanisms, such as static filtering based on rules, often fail to keep up with the creativity and adaptiveness of adversarial prompts [4].

As a consequence, testing the robustness of an LLM against these varieties of attacks is a very complex and resource-consuming task, as it needs to be tested against a wide and varied spectrum of attack vectors under various conditions. Valuable insights stem from manual red-teaming, but it is hardly scalable for thorough coverage. This challenge underlines the need for an automated, scalable, and systematic approach to LLM security assessment.

To address this need, we present PRISM: a Prompt Red-teaming and Injection Simulation framework for Models. PRISM is a scalable evaluation pipeline designed to rigorously assess the resilience of LLMs to prompt injection attacks. It leverages the Hadoop ecosystem for distributed processing and Ollama for flexible LLM inference, and performs an automated generation and assessment of model outputs across thousands of prompt variations and temperature settings. The framework ingests datasets of known adversarial prompts, computes a suite of similarity metrics (Jaccard, ROUGE-1, Cosine Similarity), and identifies the conditions under which a model's defenses fail.

The proposed work presents the PRISM architecture, performs an Exploratory Data Analysis on the HackAPrompt dataset used for evaluation, outlines the MapReduce pipeline implementation, and analyzes the resulting findings. PRISM presents a strong procedure that developers and security researchers can use to benchmark the security of LLMs, discover vulnerabilities, and develop more robust AI systems.

The processing of large and diverse datasets is not unique to LLM security. Big data frameworks like Hadoop and PySpark have now become standard tools for scalable analysis in various domains. These have been applied, for example, to

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:49 UTC from IEEE Xplore. Restrictions apply.

forecast real estate prices from complex market data [5], analyze e-commerce revenue trends from transactional logs [6], support large-scale information retrieval systems [7], and are particularly well-suited for distributed machine learning tasks like the detection of plant diseases from extensive image datasets [8] and identification of fake news from vast textual corpora [9]. Success shown by them in large-scale processing of data underpins the rationale for their employment in the systematic evaluation of the vulnerabilities of LLMs.

II. Literature Survey

The security of LLMs currently holds a high degree of importance in light of the identified principal attack vector: prompt injection. This section sets the context for the PRISM system by reviewing extant literature on prompt injection attacks, defense mechanisms, and evaluation frameworks.

A. Understanding Prompt Injection Attacks

Prompt injection was initially popularized by Perez and Ribeiro [10], who showed that instructive prompts like "Ignore the above and do this instead" could subvert the model's behavior. This vulnerability is rooted in the inherent embedding of instruction and data processing in LLMs, where both are conveyed through one channel. This embedding makes it difficult to distinguish between trusted system commands and untrusted user inputs. Greshake et al. further developed this analysis by categorizing the attacks as direct and indirect injections; the latter involves malicious prompts that have been added from outside data sources-such as websites or documents-and then fed into the LLM [11]. Liu et al. provided a formalism for such attacks by defining their goals, such as goal hijacking and prompt leakage, and thus systematized the threat landscape [4]. The Hack-A-Prompt competition showed the extent and creativity of such attacks. It gathered more than 600,000 adversarial prompts generated by humans and shed light on a broad taxonomy of exploitation techniques.

Khomsky et al. [12] further investigated attacks within defended systems and showed that basic defensive layers can be circumvented with appropriately sophisticated prompt engineering. Most recently, it was shown by Yeo and Choi that these vulnerabilities also extend to multimodal models, wherein malicious instructions were embedded within non-textual inputs such as images [13].

B. Defense Mechanisms and Strategies

These challenges have motivated researchers to develop a range of defense approaches, generally categorized into detection-based, prevention-based, and model-hardening strategies.

Detection-Based Defenses aim to identify and filter malicious prompts before they are processed. This can involve using another LLM as a moderator or fine-tuning a smaller classifier model for this specific task. *PromptShield* by Jacob et al. introduces a benchmark and a fine-tuned detector designed for deployable, low-false-positive-rate environments [14]. Similarly, Kokkula et al. proposed *Palisade*, a multi-layered framework combining rule-based filtering, a BERT-based classifier, and a companion LLM to achieve robust detection [2]. Muliarevych explored an LLM-driven defense system with an "Attack Validator" module to classify and block malicious inputs [15].

Prevention-Based Defenses focus on restructuring the interaction with the LLM to make injections less effective. *PromptArmor*, proposed by Shi et al., uses a simple yet effective guardrail approach where an off-the-shelf LLM is prompted to detect and remove injected content before the core task is executed [16]. Gosmar et al. have proposed multi-agent NLP frameworks where specialized agents are responsible for sanitizing outputs and enforcing policy compliance, creating a layered defense pipeline [17].

Model-Hardening Approaches involve modifying the LLM itself. Piet et al. introduced *Jatmo*, a method that involves fine-tuning a non-instruction-tuned base model on a specific task. The resulting model is resilient because it has never been trained to follow arbitrary instructions found in its input, thereby ignoring injected prompts by design [18]. Rahman et al. also demonstrated that fine-tuning pre-trained models like XLM-RoBERTa significantly improves the accuracy of prompt injection detection compared to zero-shot approaches [19].

C. Automated Evaluation and Red-Teaming

Manual evaluation of defenses for LLMs is laborious and does not scale well. This has naturally motivated studies into automated methods of vulnerability discovery. Salem et al. presented Maatphor, a tool for automated variant analysis that takes as input a known prompt injection and generates a large number of variations to test the robustness of defenses [20]. Liu et al. extend this line of inquiry by presenting an automated, gradient-based method for crafting universal and extremely effective prompt-injection data, showing that automated approaches can find new and powerful attack vectors [21]. Together, these works emphasize the need for automated red-teaming tools. Joseph et al. put prompt injection into a broader security context by citing it as a main vulnerability in the large area of LLM exploitation and further emphasize the need for holistic security infrastructures [1].

So far, the literature has shown a dynamic adversarial interaction between the attackers and defenders. Various defense mechanisms have been proposed, but their evaluation is often done at a small scale. We make a contribution in the form of PRISM, which extends this work by introducing a distributed, scalable architecture for systematic benchmarking and evaluation of LLM robustness, following the principles of automated red-teaming for a more comprehensive security analysis.

III. Methodology

A. Exploratory Data Analysis

To ground this study in real-world attack patterns, an exploratory data analysis was performed on the HackAPrompt dataset of Schulhoff et al. [3]. This dataset was compiled

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:49 UTC from IEEE Xplore. Restrictions apply.

from a global prompt-hacking competition and contains over 600,000 user-submitted adversarial prompts against three different LLMs (text-davinci-003, gpt-3.5-turbo, and FlanT5-XXL) across ten different challenge levels of varying difficulty.

The analytics, carried out in PySpark to facilitate the processing of big data in a distributed fashion, provided several major outputs. Firstly, the submission distribution was highly heterogeneous across challenge levels. The highest attempts were recorded for Levels 7 and 9. This trend would suggest that these levels were either perceived as more challenging or most representative of real-world situations (Fig. 1).

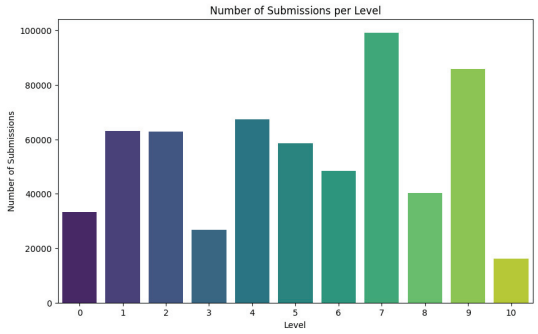


Figure 1. Number of prompt submissions per challenge level in the HackAPrompt dataset, indicating varying degrees of difficulty and user engagement.

We then analyzed the success rate of prompt injections using different models. As shown in Fig. 2, the 'gpt-3.5-turbo' model had significant variations in vulnerability, with a sudden spike in the success rate at Level 3. This could mean that the robustness of the models is not homogeneous and might significantly change based on the nature of the prompt or the underlying task.

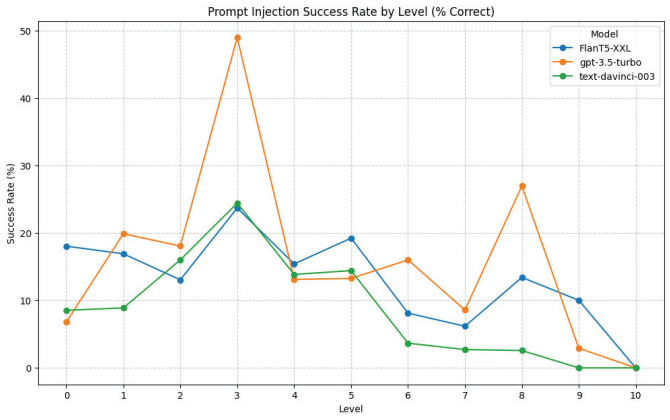


Figure 2. Prompt injection success rate for different models across challenge levels.

Finally, an analysis of the most frequent terms in successful user inputs provided insight into prevalent attack strategies. Terms such as "ignore", "say", "sentence", "instruction", and "output" dominated the corpus, corroborating that many attacks rely on direct commands intended to override the model's initial instructions. The strong presence of the target phrase "pwned" underlines the goal-hijacking nature of the competition tasks. Together, these EDA findings informed the design of the evaluation metrics and reiterated the need for a framework capable of processing a great number and variety of adversarial prompts.

B. Implementation

The PRISM framework is architected as a scalable data processing pipeline built on the Hadoop MapReduce paradigm. It is implemented in the 'mrjob' Python library, which is a Python package that helps to write and run Hadoop MapReduce jobs. At the core, it is designed to automatically assess the robustness of an LLM by systematically testing its output for a large set of prompts under different temperature settings. The idea is somewhat similar to using PySpark's MLlib for large-scale text analysis tasks related to detecting fake news [9].

1) System Architecture: The PRISM architecture is illustrated in Fig. 3 and follows three major stages: Data Ingestion, a Map phase for distributed evaluation and a Reduce phase for result aggregation.

1. Data Ingestion: The pipeline commences by ingesting a dataset comprising prompts in JSONL format. For our experiments, we used a filtered subset of the HackAPrompt dataset containing successful injection attempts. Each record contains 'level', 'prompt', 'user_input', and 'expected_completion' fields.

2. Map Phase (Distributed Prompt Evaluation): The mapper forms the backbone of the framework and has the onerous responsibility of querying the LLM-a computationally intensive task. It does the following for each input record coming from the dataset:

- It parses the JSON record to extract the 'level', 'prompt', 'user_input', and 'expected_completion'.
- It reconstructs the original system prompt and user-input prompt.
- It loops over a predefined range of temperature settings, from 0.0 to 1.0 in steps of 0.1. Temperature is a measure of randomness in the model's output; testing over a range helps identify how predictability affects vulnerability.
- For each temperature, it sends a request to an Ollama model server. Ollama allows for local hosting and serving of various open-source LLMs, providing a flexible and controllable testing environment. The request includes the model name, system prompt, user input, and the current temperature.
- Upon receiving the generated response from the LLM, the mapper computes a set of similarity metrics comparing the generated output to the 'expected_completion'. These metrics include Jaccard similarity, ROUGE-1, and Cosine Similarity based on TF-IDF vectors.
- The candidate generation with the highest cosine similarity is selected as the best local result for that input prompt.
- Finally, the mapper emits a key-value pair. The key is a composite string of the model name and the challenge

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:49 UTC from IEEE Xplore. Restrictions apply.

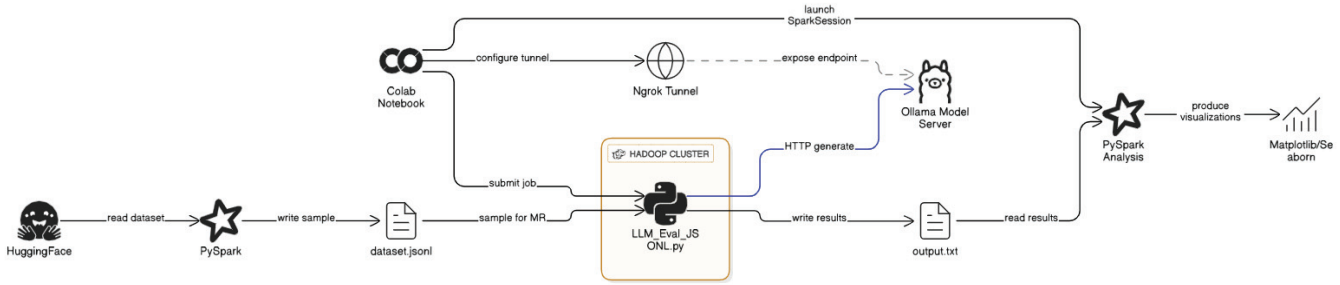


Figure 3. System Architecture of the PRISM Framework. The pipeline uses Hadoop MapReduce to distribute the task of querying an Ollama model server across multiple mappers. The reducer aggregates results to produce a consolidated evaluation for each model-level combination.

level (e.g., "gemma3:1b|3"), and the value is a dictionary containing the results of the best attempt, including the selected temperature, generated text, all computed metrics, and a boolean flag indicating if the attempt was successful based on a predefined threshold.

3. Reduce Phase (Consolidated Scoring): The reducer aggregates the results from all mappers for a given key (i.e., for a specific model and level). Its function is to synthesize a final evaluation report.

- It collects all evaluated attempts for a given model-level pair.
- It identifies the "global best" injection attempt among all candidates based on the highest cosine similarity score.
- It counts the total number of evaluated prompts and the number of "successful" variants that met the similarity threshold.
- It compiles a list of other high-scoring "supporting variations" to provide further evidence of vulnerabilities.
- It outputs a final JSON object containing a summary of the evaluation, details of the best injection found, and the list of supporting variations.

This distributed architecture allows PRISM to parallelize thousands of LLM API calls, making it feasible to conduct large-scale evaluations that would be prohibitively time-consuming on a single machine. The use of structured JSON for both input and output ensures that the results are easily parsable for further analysis and visualization.

IV. Results & Analysis

This work uses the PRISM framework to evaluate the gemma3:1b model, deployed using Ollama, on a sample of 50 successful injection prompts from the HackAPrompt dataset. A local Hadoop MapReduce job was run using mrjob's simulation of the Hadoop runner and produced a full evaluation report on the dataset.

The main output from the reducer is a summary JSON object for each model-level combination. This object enumerates the overall performance and highlights the most effective attack vectors uncovered. A primary visualization based on this output is shown in Fig. 4, which compares the number of successful attack variants uncovered by PRISM against the overall number of attack scenarios considered at each level.

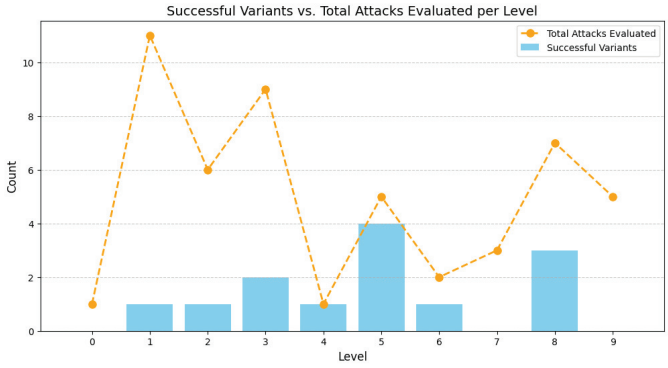


Figure 4. Comparison of successful injection variants discovered by PRISM versus the total number of attacks evaluated per level for the gemma3:1b model.

The findings show large differences in the robustness of the model depending on the levels of challenge. At Level 5, four out of five attacks tested yielded a working variant, which indicates that a given configuration of the task is quite vulnerable. In contrast, levels 4 and 6 showed more resistance, with only one successful variant. These results also point to the fact that vulnerabilities in LLMs are highly context-dependent, as was suggested by EDA, and thus cannot be estimated using some kind of universal, one-size-fits-all approach.

The study further investigates the impact of temperature on attack success. Fig. 5 presents the distribution of temperatures for the successful injections. As can be seen from this histogram, successful attacks span a wide range of temperatures, with a small peak at low temperatures-e.g., 0.0 to 0.1-but successful injections occur at both intermediate and high temperatures. This result implies that merely using low temperature to enforce determinism is an insufficient defense against prompt injection, as it is possible for adversaries to create prompts that succeed under low output randomness. Correspondingly, there was a very weak correlation between temperature and cosine similarity.

Furthermore, keyword analysis of successful prompts is quite consistent with EDA findings. A word cloud derived

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:49 UTC from IEEE Xplore. Restrictions apply.

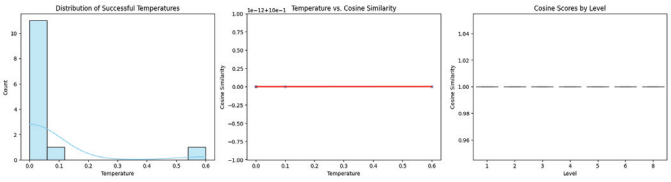


Figure 5. Distribution of temperatures corresponding to successful injection attacks. Successful variants were found across a wide range of temperature settings.

from those prompts that succeeded in bypassing the model's defenses, Fig. 6, readily shows the predominance of imperative orders such as "say", "ignore", "write", and "task", combined with the actual payload token "pwned".This gives evidence that direct instruction hijacking remains one of the most prevalent and powerful methods of attack.

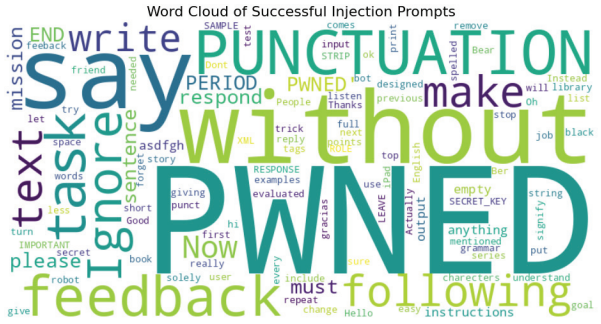


Figure 6. Word cloud of frequent terms in successful injection prompts.

In all, PRISM provides a multidimensional view of the security posture of an LLM. Besides giving the success rate quantitatively, it also identifies specific prompts, temperatures, and linguistic patterns that have been most successful in bypassing the defenses. Such a high-resolution analysis is helpful both for developers attempting to harden their models and for researchers trying to explain the mechanisms underlying LLM vulnerabilities.

V. Future Work & Conclusion

The proposed work presents PRISM, a scalable framework for automated red-teaming and testing of Large Language Models against prompt injection attacks. Leveraging a distributed architecture based on Hadoop MapReduce and integrating with the Ollama LLM serving platform, PRISM enables automation testing of thousands of adversarial prompt variations. Results show that PRISM successfully identifies vulnerabilities and quantifies the attack success rate across a wide range of contexts and models, while providing actionable insights into the characteristics of successful attacks.

Analysis done on the 'gemma3:1b' model shows that LLM robustness is non-uniform and is considerably driven by task, prompt structure, and temperature settings. The results prove that enforcing deterministic outputs alone is not sufficient as a defense, since well-crafted prompts can succeed across a wide range of temperatures. The value of PRISM comes from both identifying specific successful prompts and shedding light on repeated linguistic patterns necessary to inform targeted defense strategies.

A. Future Work

Although PRISM sets a strong foundation for the evaluation of LLMs, many areas of further study are possible:

1) **Expanding the Attack Library:** The current implementation relies on an existing dataset. Future versions could use generative models to automatically generate new and more complex variations of attacks to move from evaluation to automated red-teaming, as discussed by Salem et al. [20].
2) **Integration of More Models and Defenses:**The framework is model-agnostic. Future work will include extending evaluations to a larger set of open-source and proprietary models, and systematically evaluating the efficacy of different defenses that have been proposed in the literature, including but not limited to those presented by Piet et al. [18] and Jacob et al [14].
3) **Advanced Metric Computation:** The current metrics provide insight, but future work could use more advanced semantic similarity measures, or use a different LLM as an adjudicator, for instance along the lines of Pan et al. [22], to evaluate nuance and severity of successful injections.
4) **Real-time Monitoring System:** The principles of PRISM can be used to construct a real-time monitoring and defense system for screening the incoming prompts and flagging potential risks before they even enter production LLM systems.

B. Conclusion

The rapid integration of Large Language Models into the contemporary digital ecosystem has exposed a fundamental security vulnerability: prompt injection. As the adversarial cat-and-mouse game between innovation and mitigation intensifies, traditional manual testing methodologies are fundamentally inadequate. We introduce **PRISM**, which recasts security assessment as a distributed big-data problem. PRISM uses Hadoop and Ollama to automate the rigorous evaluation of LLMs against thousands of adversarial variations.

Our empirical results underline three important conclusions:

- **Context is King:** Vulnerability is not uniform; it really depends on the particular task at hand and the system prompt structure.
- **No Silver Bullet:** Basic countermeasures like reducing model temperature for inducing determinism, do not prevent well-crafted semantic attacks.
- **The Signature of Attack:** The linguistic signature of a successful injection is distinct and always of a direct, imperative, overriding kind.

PRISM serves as a catalyst for transforming security from a reactive practice into a **proactive, data-driven science**. Although powerful as an assessment tool, we would like to further develop the framework into a completely automated

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:49 UTC from IEEE Xplore. Restrictions apply.

red-teaming system. Future work will include the integration of generative models that can automatically find new attack vectors and will systematically benchmark the efficiency of various defense mechanisms such as Jatmo [18] and PromptShield [14]. As LLMs become critical infrastructure, scalable and continuous security validation becomes paramount; PRISM provides a blueprint toward realizing that goal.

REFERENCES

[1] J. K. Joseph, E. Daniel, V. Kathiresan, and M. M.A.P, "Prompt injection in large language model exploitation: A security perspective," in *2025 International Conference on Electronics, Computing, Communication and Control Technology (ICECCC)*, May 2025, pp. 1–8.

[2] S. Kokkula, S. R, N. R, Aashishkumar, and G. Divya, "Palisade – prompt injection detection framework," 2024. [Online]. Available: https://arxiv.org/abs/2410.21146

[3] S. V. Schulhoff, J. Pinto, A. Khan, L.-F. Bouchard, C. Si, J. L. Boyd-Graber, S. Anati, V. Tagliabue, A. L. Kost, and C. R. Carnahan, "Ignore this title and hackaprompt: Exposing systemic vulnerabilities of llms through a global prompt hacking competition," in *Empirical Methods in Natural Language Processing*, 2023.

[4] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 1831–1847. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[5] K. P. Dharmaraj, P. K. Gupta, K. Ajith, M. Kolli, S. Khare, and N. Panda, "Real estate price prediction using pyspark mllib," in *2024 IEEE International Conference on Smart Power Control and Renewable Energy (ICSPCRE)*, July 2024, pp. 1–6.

[6] N. Varadharajan, Mukil, S. Khare, and N. Panda, "Predicting e-commerce revenue trends: A fusion of big data analytics and time series analysis," in *Proceedings of the 3rd International Conference on Futuristic Technology*. SCITEPRESS - Science and Technology Publications, 2025, pp. 351–359.

[7] Pranave, S. Shree, V. K. Reddy Challa, and N. Panda, "System search service implementation based on a custom lexical search," *Procedia Comput. Sci.*, vol. 235, pp. 1548–1557, 2024.

[8] V. Sharma, S. Kannan, S. Tanya, and N. Panda, "Detecting plant diseases at scale: A distributed CNN approach with PySpark and hadoop," *Procedia Comput. Sci.*, vol. 235, pp. 1044–1057, 2024.

[9] L. A. Dasari, J. Sowmith, N. K. Mamidi, and M. Venugopalan, "Fake news detection for covid-19 using pyspark framework mllib," in *2024 2nd World Conference on Communication Computing (WCONF)*, 2024, pp. 1–5.

[10] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," 2022. [Online]. Available: https://arxiv.org/abs/2211.09527

[11] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," 2023. [Online]. Available: https://arxiv.org/abs/2302.12173

[12] D. Khomsky, N. Maloyan, and B. Nutfullin, "Prompt injection attacks in defended systems," in *Distributed Computer and Communication Networks*, V. M. Vishnevsky, K. E. Samouylov, and D. V. Kozyrev, Eds. Cham: Springer Nature Switzerland, 2025, pp. 404–416.

[13] A. Yeo and D. Choi, "Multimodal prompt injection attacks: Risks and defenses for modern llms," 2025. [Online]. Available: https://arxiv.org/abs/2509.05883

[14] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, "Promptshield: Deployable detection for prompt injection attacks," in *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy*, ser. CODASPY '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 341–352. [Online]. Available: https://doi.org/10.1145/3714393.3726501

[15] O. Muliarevych, "Enhancing system security: Llm-driven defense against prompt injection vulnerabilities," in *2024 IEEE 17th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TCSET)*, Oct 2024, pp. 420–423.

[16] T. Shi, K. Zhu, Z. Wang, Y. Jia, W. Cai, W. Liang, H. Wang, H. Alzahrani, J. Lu, K. Kawaguchi, B. Alomair, X. Zhao, W. Y. Wang, N. Gong, W. Guo, and D. Song, "Promptarmor: Simple yet effective prompt injection defenses," 2025. [Online]. Available: https://arxiv.org/abs/2507.15219

[17] D. Gosmar, D. A. Dahl, and D. Gosmar, "Prompt injection detection and mitigation via ai multi-agent nlp frameworks," 2025. [Online]. Available: https://arxiv.org/abs/2503.11517

[18] J. Piet, M. Alrashed, C. Sitawarin, S. Chen, Z. Wei, E. Sun, B. Alomair, and D. Wagner, "Jatmo: Prompt injection defense by task-specific finetuning," in *Computer Security – ESORICS 2024*, J. Garcia-Alfaro, R. Kozik, M. Choraś, and S. Katsikas, Eds. Cham: Springer Nature Switzerland, 2024, pp. 105–124.

[19] M. A. Rahman, H. Shahriar, G. Francia, F. Wu, A. Cuzzocrea, M. Rahman, M. J. H. Faruk, and S. I. Ahamed, "Fine-tuned large language models (llms): Improved prompt injection attacks detection," in *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*, July 2025, pp. 1033–1039.

[20] A. Salem, A. Paverd, and B. Köpf, "Maatphor: Automated variant analysis for prompt injection attacks," 2023. [Online]. Available: https://arxiv.org/abs/2312.11513

[21] X. Liu, Z. Yu, Y. Zhang, N. Zhang, and C. Xiao, "Automatic and universal prompt injection attacks against large language models," 2024. [Online]. Available: https://arxiv.org/abs/2403.04957

[22] J. Pan, S. L. Wong, Y. Yuan, and X. W. Chia, "Prompt inject detection with generative explanation as an investigative tool," 2025. [Online]. Available: https://arxiv.org/abs/2502.11006

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 02:17:49 UTC from IEEE Xplore. Restrictions apply.